UNIVERSITÉ LIBRE DE BRUXELLES

Erasmus Mundus Joint Master in
Data Engineering and Artificial Intelligence

TPC-DS on MariaDB

Implementation and Experimental Evaluation

INFO-H-419 — Data Warehouses
Part I project report

[Group number]

Group members	Student IDs
[Full name 1]	*[Student ID 1]*
[Full name 2]	*[Student ID 2]*
[Full name 3]	*[Student ID 3]*
[Full name 4]	*[Student ID 4]*

Course lecturer: [Lecturer name]

October 2026

Abstract

This project implements and evaluates TPC-DS analytical queries on MariaDB. The implementation combines the official v4.0.0 data generator, a relational retail warehouse, MariaDB SQL adaptations and a Docker execution workflow. All 99 query identifiers are supported, with 35 adopted query optimization variants integrated into the complete workload.

The evaluation includes complete original and adopted workloads at scale factors 1, 2, 3, 4, using a 2 GiB InnoDB buffer pool. The eight passes execute 792 query files. At SF4, the original workload completed in 2 921.5 seconds and the adopted workload in 1 536.4 seconds. Result comparison confirms matching values for all 99 queries at each scale. Recorded runtime reductions range from 25.3% to 49.0% across the four sizes. Repeated local SF1 experiments complement these measurements with per-query medians, execution plans and targeted semantic tests.

The report presents the implementation pipeline, result-validation procedure, physical-design experiments and representative SQL transformations. These include predicate pushdown, conditional aggregation, late dimension joins and window-query restructuring. The results illustrate how analytical query structure, access paths and memory settings affect execution time. The source package preserves the executed SQL, answers, timing records and replication commands used to produce the reported measurements.

Contents

1 Introduction **1**

1.1 Starting point and project contributions 1

1.2 Benchmark scope 1

2 Technology and warehouse implementation **1**

2.1 MariaDB and InnoDB 1

2.2 Retail model and schema 2

2.3 Data generation and loading 2

2.4 Query preparation and MariaDB adaptation 2

3 Workloads and experimental methodology **3**

3.1 Query coverage and optimization count 3

3.2 Experimental configurations 3

3.3 Timing and aggregation 3

4 Correctness checks and result validation **4**

4.1 Complete answer comparison 4

4.2 NULL-safe hierarchy translation 4

4.3 Multiple result sets and row accounting 5

5 Optimization techniques and representative queries **5**

5.1 Q5: predicate pushdown into union branches 5

5.2 Q4 and Q31: conditional aggregation by period 5
5.3 Q74: pivot channel/year groups 6
5.4 Q39: late dimension joins and access order 6
5.5 Q51: combine daily keys and preserve windows 6
5.6 Q67 and MariaDB optimizer behavior 6
5.7 Repeated local measurements 7

6 Complete-workload results 7
6.1 Original and adopted workloads 7
6.2 Dataset-size comparison 9
6.3 Where the aggregate time is saved 10
6.4 How individual query behavior changes with scale 11

7 Physical-design and automation work 11
7.1 Indexing and statistics 11
7.2 Reproducible execution workflow 13
7.3 Report and evidence packaging 13

8 Conclusion 14

A Adopted optimization query IDs 16

B Full runtime table across the four scales 16

C Implementation and evidence map 18

1 Introduction

Data warehouses organize historical business data for analytical processing. Their workloads typically join large fact tables with descriptive dimensions and aggregate measures over substantial portions of the database. Query performance therefore depends on the interaction between logical SQL, physical design, optimizer decisions and the available memory budget.

The INFO-H-419 project applies these concepts through an implementation of TPC-DS. Our objectives are to execute the benchmark's analytical workload on MariaDB, validate the results of the SQL adaptations, study optimization techniques, and measure execution time at different dataset sizes.

1.1 Starting point and project contributions

The implementation builds on the `mariadb-tpcds` repository [7], including its toolkit integration, schema, loader, adapted query instances and earlier experiments. The project also draws on `tpcds-run-tool` [8]. These resources provided an operational starting point for the subsequent engineering and evaluation.

The completed work comprises SQL optimization variants, controlled local experiments, NULL-safe rollup translations, full answer capture, complete 99-query Docker execution, configurable memory settings, explicit data seeds and preservation of execution artifacts. The contributed query directories and source headers record the authorship and selected versions.

This report follows the technology–implementation–evaluation structure used in earlier course reports on Oracle and PostgreSQL [1, 2]. Its measurements come from the MariaDB execution artifacts described below.

1.2 Benchmark scope

TPC-DS models the analytical workload of a retail supplier through its data model and query templates [9]. The implementation uses these data and queries for a course experiment. Its execution procedure runs complete query files serially and reports elapsed time in seconds, together with per-query answer comparisons and the corresponding database settings.

2 Technology and warehouse implementation

2.1 MariaDB and InnoDB

MariaDB provides the relational query engine and InnoDB supplies the table storage. The workload exercises joins, exact decimal arithmetic, grouped aggregation, common table expressions (CTEs), rollups and window functions. Docker packages the database and toolchain, while Python manages execution, timing, result serialization and analysis.

The complete Docker workloads use MariaDB 13.0.2. The initial local development experiments use MariaDB 10.11.14. Their results are presented as separate series. InnoDB's buffer pool caches database pages [3]; temporary tables, join buffers and sort buffers provide additional workspace for query execution. The Docker measurements use a 2 GiB buffer pool, 32 KiB pages and a persistent statistics sample setting of 160.

2.2 Retail model and schema

`sql/01_schema.sql` defines 25 tables, 429 columns and 24 primary keys. Seven fact tables represent sales, returns and inventory. Seventeen shared dimensions describe products, customers, dates, locations and related business attributes; the remaining table records generator metadata.

Process	Fact tables	Example dimensions
Store	`store_sales`, `store_returns`	date, time, customer, item, store
Catalog	`catalog_sales`, `catalog_returns`	date, item, customer, call center, shipping
Web	`web_sales`, `web_returns`	date, item, customer, web site/page
Inventory	`inventory`	date, item, warehouse

Table 1: Business processes represented in the implemented warehouse.

Sales facts have transaction-line grains. Inventory is identified by date, item and warehouse. Shared dimensions connect the three channels, producing a constellation of related star and snowflake structures. The schema uses `DECIMAL` measures, `DATE` columns, UTF-8 text and explicit collations.

The schema reconciliation in `report/04-reconcile-discrepancies.md` resolves differences between toolkit files and the specification, including the customer's last-review date key and the spelling of the store tax column. `sql/02_foreign_keys.sql` describes 107 relationships. The saved local SF1 integrity check evaluated all 107 and returned zero orphan rows.

2.3 Data generation and loading

The bundled v4.0.0 `dsdgen` generates pipe-delimited data with an explicit scale and `RNGSEED=10`. The loader in `sql/04_load_data.sql` consumes the trailing delimiters and converts nullable fields using `NULLIF`. This preserves the generator's representation of NULL values [9, Clause 3.4.5].

Named Docker volumes retain generated files and the loaded database. Generation markers identify reusable datasets; later runs against the same volume reuse the loaded warehouse. Schema creation, data loading, index construction and statistics refresh precede query execution.

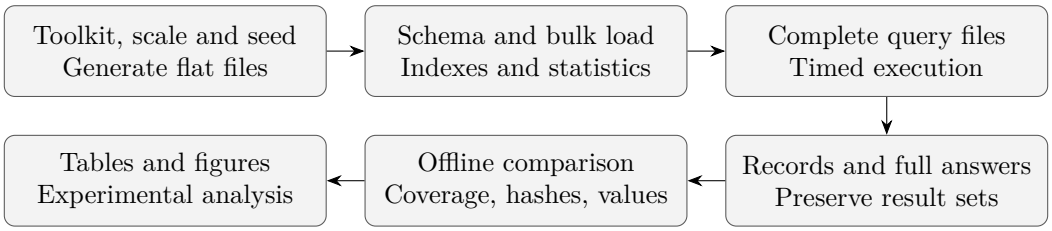


Figure 1: The implemented generation, execution and analysis workflow.

2.4 Query preparation and MariaDB adaptation

`queries/mariadb/` contains the 99 adapted originals. The preparation layer handles date intervals, derived-table aliases, MariaDB rollup syntax and full-outer-join emulation. Full joins are expressed through a left-join branch and an unmatched right-join branch combined with `UNIONALL`, preserving duplicate rows.

The execution session enables `IGNORE_SPACE` and `PIPES_AS_CONCAT`. The latter preserves concatenated identifiers in queries such as Q5. Each complete file runs in one connection, retaining its

session settings and variables across statements. Query literals are stored in the committed SQL; `RNGSEED` controls generation of the warehouse data.

3 Workloads and experimental methodology

3.1 Query coverage and optimization count

Selector	Workload	Query IDs
`all`	Complete adapted original workload	99
`all-tuned`	Complete workload with adopted optimization variants	99
`tier45`	Original versions of the selected expensive queries	40
`tuned`	Selected queries using the adopted versions	40

Table 2: Query selection through `QUERYSET`.

The adopted workload integrates 35 optimization variants among 40 selected query IDs. Five selected queries retain their adapted originals: Q38, Q58, Q66, Q71 and Q77. The remaining 59 IDs also use adapted original SQL. Thus the complete workload combines 35 selected optimization variants with 64 adapted originals. Appendix A lists the optimization IDs.

The count is generated from the selected source headers. Multiple development versions of one query contribute one query ID. Index-only contributions and compatibility fixes are tracked separately from the adopted SQL variants.

3.2 Experimental configurations

Series	Database configuration	Measurement procedure
Local SF1 studies	MariaDB 10.11.14; Ryzen 5 3500U; 6.7 GiB host RAM; 2 GiB pool; 16 KiB pages; 16 MiB temporary-table limits	Three ordinary trials per selected version and profile; full answers and saved plans
Docker workloads	MariaDB 13.0.2; 2 GiB pool; 32 KiB pages; statistics sample 160; seed 10	One serial pass per original/adopted workload; all 99 query files

Table 3: The development and complete-workload experiment series.

The Docker transcript records eight logical CPUs and approximately 6.7 GiB capacity. Run summaries retain the buffer-pool size, page size, statistics sample, workspace settings, index count and workload selection. At SF1, SF2 and SF3, both workloads record 190 total indexes. At SF4, the original records 190 and the adopted workload 188. The timing ratios in this report compare these recorded original and adopted configurations.

Executed SQL is identified by SHA-256. The preserved bundles include the exact query versions, including the Q22 revision used by the SF4 adopted run. Summary hashes connect the report's numeric tables to their input artifacts.

3.3 Timing and aggregation

`docker/bench.py` measures wall-clock time around the MariaDB client call. The interval includes connection/session setup, every statement in the query file and complete XML result transfer. Parsing, hashing and writing JSON occur after the timer. Generation, loading, index construction

and statistics refresh are measured separately from query execution. The protocol is recorded as `whole-file-xml-v1`.

For scale S, workload W and configuration C, the serial workload total is

$$T(S, W, C) = \sum_{q=1}^{99} t(q, S, W, C). \tag{1}$$

The reported original/adopted ratio is

$$R(S) = \frac{T(S, W_o, C_o)}{T(S, W_a, C_a)}. \tag{2}$$

Docker query times are recorded to 0.1 seconds. The local studies retain unrounded individual times, exclude warm-up/probe executions and report three-run medians and ranges. Instrumented plan diagnostics are stored as a separate class of execution.

4 Correctness checks and result validation

4.1 Complete answer comparison

The runner saves each query's complete answers using `result-sets-json-v1`. The representation preserves result-set boundaries, column order, repeated column names, duplicate rows, NULLs, empty strings and exact returned numeric text.

Artifact checks verify coverage of IDs 1–99, successful execution status, answer hashes and recorded row counts. The offline comparator then compares original and adopted answers on the same declared scale and seed. The four pairs provide 396 query-result comparisons: 395 identical ordered outputs and 1 reviewed permutation within equal ordering keys. All 396 query results have matching values.

The ordered `md5` and within-result-set row-sorted `md5_sorted` fingerprints support diagnosis of value and ordering differences. Comparisons retain the serialization format and dataset identity. Complete saved values are available for inspection alongside the fingerprints.

At SF2, Q71 returns the same 1,895 rows in both runs. Four row positions differ within two tied groups. Its ordering is `ext_priceDESC,i_brand_id`; hour, minute and brand text are outside those ordering keys. The review checks the identical executed SELECT, duplicate-preserving value equality, the full sort-key sequence and the absence of a row limit. Both sequences satisfy the specified ordering, including NULL prices at the end of descending order [5]. This reviewed result is retained alongside the original comparator status in the evidence inventory.

4.2 NULL-safe hierarchy translation

The rollup translations for Q36, Q70 and Q86 distinguish genuine NULL keys from subtotal markers. Leaf, subtotal and grand-total aggregates are constructed explicitly with `UNIONALL`, and each hierarchy level is assigned before ranking. Duplicate-looking rows at different levels remain separate. Adopted Q70 preserves the original state-selection and logical date filters.

Twelve live cases on MariaDB 10.11.14 checked these three adapted originals and adopted Q70 against an independent grouping/ranking oracle. The inputs cover genuine NULL grouping keys, empty relations and all-NULL measures. Manual patches preserve the translations through query regeneration.

4.3 Multiple result sets and row accounting

The complete workload produces 103 result sets across 99 query files. The SF4 Q14 output comprises two sets of 100 rows; Q39 comprises 417 and 8 rows. The structured format records each set separately and counts data rows directly. The same representation is used for the comparisons and evidence exports.

5 Optimization techniques and representative queries

The optimization work addresses repeated access, excessive intermediate rows, wide grouping keys and access-path selection. Table 4 groups the techniques and their representative query instances.

Technique	Change in query processing	Examples
Predicate pushdown	Apply selective dates inside union branches	Q5
Conditional aggregation	Compute period/channel values in grouped columns	Q4, Q31, Q74
Late dimension joins	Aggregate narrow fact keys before descriptive lookups	Q39
Window restructuring	Combine daily keys and retain cumulative operations	Q51
Bounds and plan control	Expose date ranges and retain a selected access order	Q67, Q72

Table 4: Optimization families represented in the project.

5.1 Q5: predicate pushdown into union branches

Q5 combines sales and returns from three channels before aggregating by channel and entity. The selected version joins filtered dates inside all six branches, reducing the rows passed to the union and subsequent aggregation. It retains the original rollup, final ordering and `LIMIT100`.

Each branch uses its own event date. In the web-return branch, the restriction applies to the return date, while the lookup of the original sale retains its existing join. A return during the reporting period can therefore refer to a sale from an earlier period. The inclusive interval from 28 August to 11 September contains 15 dates.

The local indexed median changes from 112.735 to 10.144 seconds. The saved plan places filtered dates inside the union branches. A nonempty fixture checks the interval endpoints, older sales, unmatched returns, duplicate business IDs, NULL measures and the rollup result.

5.2 Q4 and Q31: conditional aggregation by period

Q4 compares two years across store, catalog and web channels. Its adopted version aggregates both years per channel and joins customer descriptions afterward. The saved local plans show three sales-table nodes in place of six.

Listing 1: Q4 conditional-aggregation pattern. The full query retains the original financial expression and denominator conditions.

```
SELECT ss_customer_sk AS customer_sk,
       SUM(CASE WHEN d_year = 2001 THEN measure END) AS y1,
       SUM(CASE WHEN d_year = 2002 THEN measure END) AS y2
FROM store_sales JOIN date_dim ON ss_sold_date_sk = d_date_sk
WHERE d_year IN (2001, 2002)
GROUP BY ss_customer_sk
HAVING COUNT(DISTINCT d_year) = 2;
```

The year-presence condition retains both required groups. Conditional sums preserve NULL behavior, and positive first-year totals retain the division conditions. The local customer check found 100,000 rows, 100,000 distinct business IDs and zero NULL business IDs, supporting the surrogate-key grouping used by the tested rewrite.

Q31 applies the same principle to quarterly aggregates. It pivots three existing quarter totals into columns using `MAX(CASE...)` and then joins the channel rows. The saved actual primary-key plans show one store and one web fact scan in place of three scans of each table. Its indexed median changes from 46.947 to 15.957 seconds.

5.3 Q74: pivot channel/year groups

The contributed Q74 variant forms store and web yearly groups, pivots the four channel/year combinations into columns, and compares year-to-year ratios. Presence counters retain the four required groups and first-year conditions guard the divisions. The original `MAX(net_paid)` measure is preserved.

The full SF4 comparison returned the same ordered answers for both versions. Its original/adopted times are 64.1 and 66.8 seconds. Together with the other queries, this contributes to the complete workload measurements in Section 6.

5.4 Q39: late dimension joins and access order

Q39 calculates inventory variability by warehouse, item and month. The selected variant first aggregates inventory on narrow keys, then performs item and warehouse lookups. `STRAIGHT_JOIN` retains the tested source-first order. Both original SELECT statements, their thresholds and their output ordering are preserved.

The local indexed original median is 81.236 seconds; the selected v6 median is 0.817 seconds. Saved plans connect this change to avoiding the expensive warehouse/item-driven indexed access path. In the primary-key-only study, the original median is 3.280 seconds. These two physical-design profiles illustrate how access-path selection changes the cost of the same logical query.

5.5 Q51: combine daily keys and preserve windows

Q51 compares cumulative store and web sales over dates. The selected version creates tagged daily channel groups, combines their keys, and computes the running operations over the resulting daily series. Channel-presence flags retain the distinction between an absent channel and a present NULL-valued group.

The rewrite preserves both the running sum and the later running maximum, including their behavior when increments are negative. The fixture exercises channel-only dates, missing periods and negative increments. The local indexed median changes from 60.528 to 48.549 seconds, with the original final ordering and limit retained.

5.6 Q67 and MariaDB optimizer behavior

The contributed Q67 variant resolves a date-key interval into session variables and filters `store_sales` using those bounds. It retains the rollup and ranking structure and specifies its session workspace settings. Executing the whole file in one connection retains the variables and settings through the final SELECT.

The plan studies examine derived-table merging and split/lateral materialization [4, 6]. Saved EXPLAIN and instrumented outputs record how these transformations affect repeated aggregate evaluation. The Q4/Q67 follow-up in `report/27-query-followup.md` also compares later candidates against the adopted versions. Q4 v4 measured approximately 1.85–1.88 times faster than v2 in those local comparisons; the complete SF4 workload uses the adopted v2.

5.7 Repeated local measurements

Query	Variant	Original median (s)	Candidate median (s)	Ratio
Q4	v2	54.866	15.209	3.61
Q5	v1	112.735	10.144	11.11
Q31	v2	46.947	15.957	2.94
Q39	v6	81.236	0.817	99.44
Q51	v1	60.528	48.549	1.25

Table 5: Local SF1 medians of three ordinary runs per version with 117 visible secondary indexes plus primary keys. All retained answers matched. Q39 uses its final correction series.

The raw trial values, job order, saved plans, result comparisons, configuration snapshots and SQL hashes are preserved in `experiments/evidence/riccio_sf1_20261014/`. These measurements describe the native MariaDB 10.11.14 development environment in Table 3.

6 Complete-workload results

6.1 Original and adopted workloads

Scale	Original (s)	Adopted (s)	Ratio	Saved (%)	Answers	Indexes (O/A)
1	431.4	219.9	1.96	49.0	99/99	190/190
2	1 102.4	785.0	1.40	28.8	99/99*	190/190
3	1 765.3	1 318.0	1.34	25.3	99/99	190/190
4	2 921.5	1 536.4	1.90	47.4	99/99	190/188

Table 6: Complete workload measurements. Each workload executes all 99 query files. Ratio is original/adopted total time; Saved is the percentage reduction. Answers compares complete result values. O/A denotes the recorded total index counts. The SF2 marker identifies Q71's reviewed tied-row ordering.

All eight measured workloads completed their 99 query files with zero SQL errors. The total-time ratios range from 1.34 to 1.96, with the adopted workload recording lower aggregate time at every scale. The percentage reduction is 25.3%–49.0%. These totals include every query, including the 64 IDs without a selected SQL optimization variant.

At SF4, the original workload completed in 2 921.5 seconds and the adopted workload in 1 536.4 seconds. Both completed 99 query files with zero SQL errors, and all 99 ordered saved answers matched. The recorded total-time ratio is approximately 1.90, a reduction of 1,385.1 elapsed seconds between the two passes.

Q27 records 8.0 seconds original and 24.3 adopted; Q66 records 0.9 and 30.4 seconds; Q74 records 64.1 and 66.8 seconds. Q66 retains the original SQL logic, illustrating the role of database access

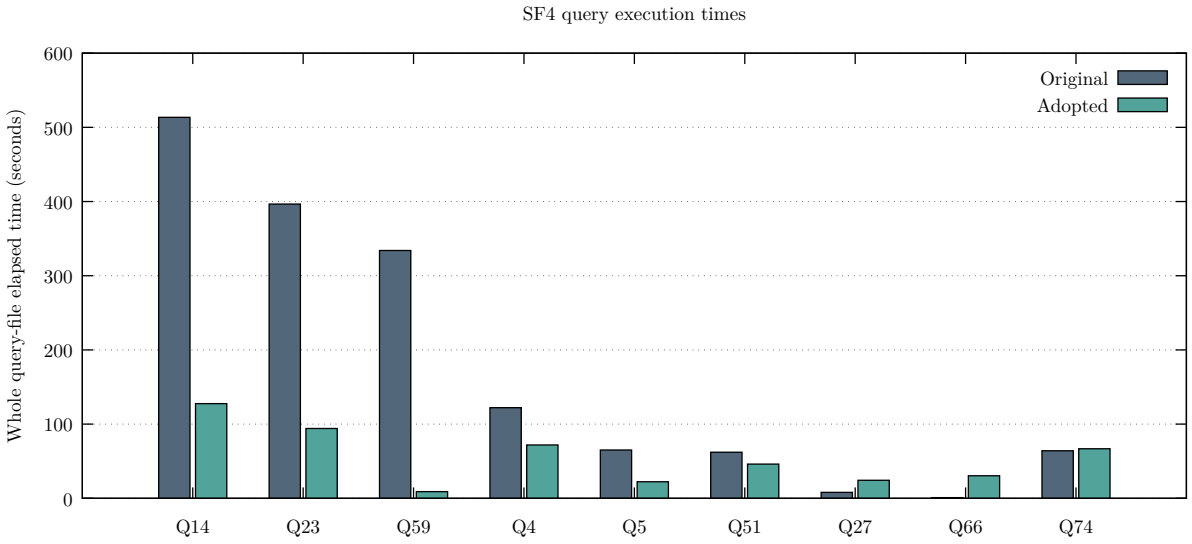


Figure 2: Selected original and adopted SF4 query execution times.

Query	Original (s)	Adopted (s)	Observed t_o/t_a
Q14	513.5	127.6	4.02
Q23	396.6	94.1	4.21
Q59	334.0	9.0	37.11
Q4	122.1	71.9	1.70
Q5	65.1	22.3	2.92
Q39	10.1	3.0	3.37
Q51	62.1	46.1	1.35
Q27	8.0	24.3	0.33
Q66	0.9	30.4	0.03
Q74	64.1	66.8	0.96

Table 7: Representative SF4 timings and their original/adopted ratios.

configuration alongside query rewrites. Appendix B contains all 99 query times at every scale; per-query CSVs accompany the LaTeX sources.

6.2 Dataset-size comparison

The complete Docker pairs cover scale factors 1, 2, 3, 4. The memory budget is held at a 2 GiB buffer pool with 32 KiB pages and a statistics sample setting of 160. SF4 serves as the reference size. SF1 is one quarter, SF2 one half and SF3 three quarters of its nominal generator scale, providing three smaller sizes on the same measurement machine.

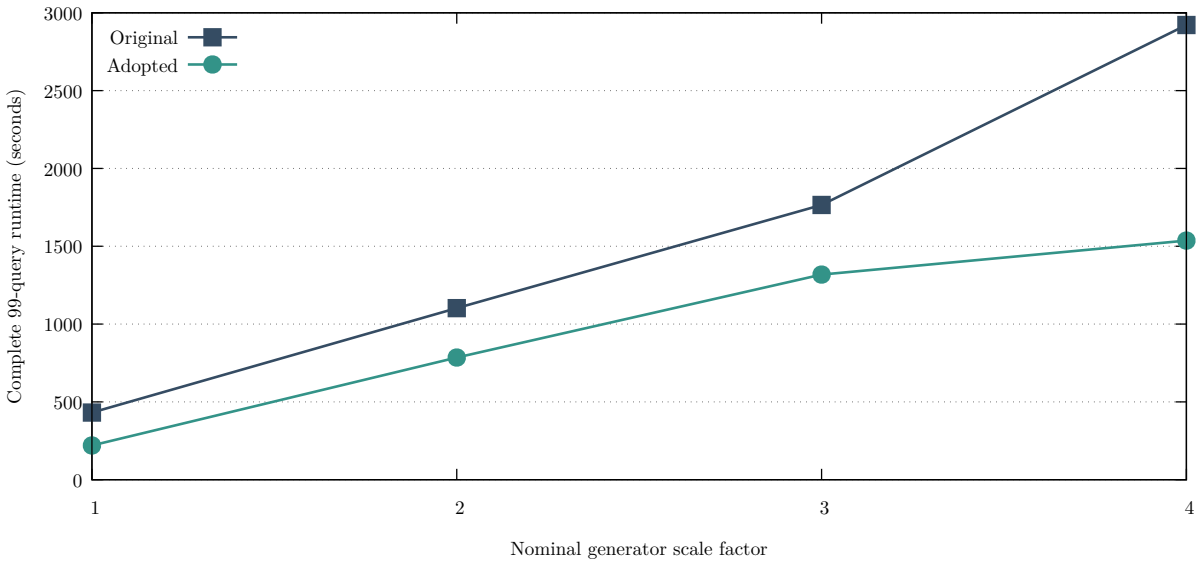


Figure 3: Total execution time of all 99 query files for the original and adopted workloads. Markers identify the measured scale factors.

From SF1 to SF4 the observed original total grows by 6.77 times for a fourfold increase in nominal generator scale. The executed query-file hashes agree across these two scales.

From SF1 to SF4 the observed adopted total grows by 6.99 times for a fourfold increase in nominal generator scale. Executed SQL hashes differ for Q22; the corresponding executed versions are preserved in the evidence bundles.

Scale step	Nominal multiplier	Original time multiplier	Adopted time multiplier
SF1–SF2	2.00	2.56	3.57
SF2–SF3	1.50	1.60	1.68
SF3–SF4	1.33	1.65	1.17

Table 8: Changes between adjacent measured scale factors. Each runtime multiplier divides the later workload total by the previous one.

The largest relative step for the adopted workload is SF1–SF2, where the workload increases from 219.9 to 785.0 seconds, a factor of 3.57. Its smallest relative step is SF3–SF4, from 1,318.0 to 1,536.4 seconds, a factor of 1.17. The original series grows from 431.4 to 2,921.5 seconds across the four sizes. The recorded benefit therefore varies with scale: the middle-size ratios are 1.40 and 1.34, compared with 1.96 at SF1 and 1.90 at SF4.

The normalization starts each series at one. The adopted series begins from 219.9 seconds, while the original begins from 431.4 seconds; its larger relative increase from that lower baseline coexists

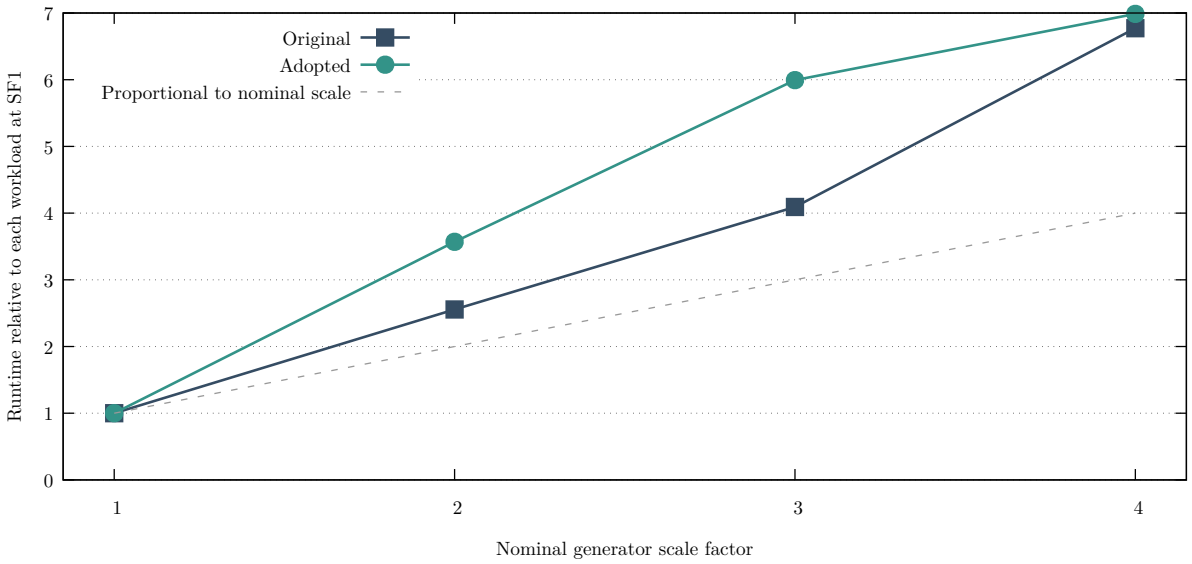


Figure 4: Each workload normalized to its own SF1 runtime. The dashed line shows growth proportional to nominal generator scale.

with lower absolute elapsed time at every measured scale.

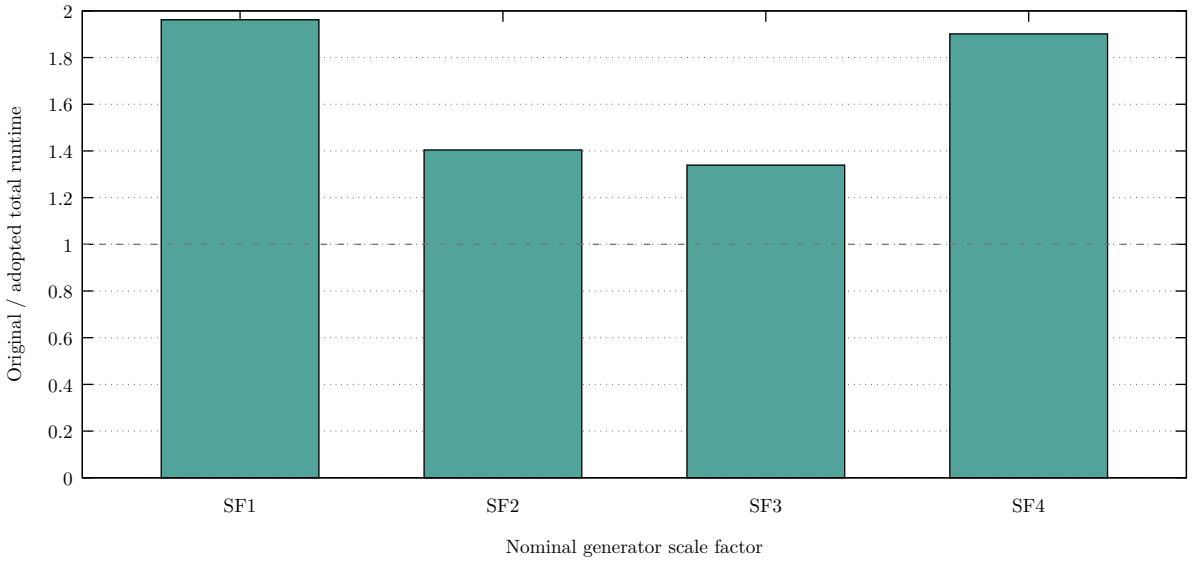


Figure 5: Recorded original/adopted total-runtime ratios. The horizontal reference at one denotes equal total execution time.

6.3 Where the aggregate time is saved

Q14, Q23 and Q59 are the three largest contributors to elapsed-time savings at every scale. Together they account for approximately 41–43% of original workload time and 14–15% of adopted workload time.

At SF3, these three queries save 534.8 seconds, while the net 99-query saving is 447.3 seconds. The remaining queries collectively offset 87.5 seconds of that reduction. At SF4, the three save 1,013.4 of the net 1,385.1 seconds. This illustrates why the full-workload total and individual-

Scale	Original (s)	Adopted (s)	Saved (s)	Original (%)	Adopted (%)
1	179.3	30.4	148.9	41.6	13.8
2	449.9	109.7	340.2	40.8	14.0
3	729.4	194.6	534.8	41.3	14.8
4	1 244.1	230.7	1 013.4	42.6	15.0

Table 9: Combined Q14/Q23/Q59 execution time, time saved and share of each complete workload's runtime.

query counts provide complementary views of performance.

Scale	Lower time	Higher time	Same rounded time	Net saved (s)
1	42	23	34	211.5
2	27	65	7	317.4
3	22	69	8	447.3
4	57	30	12	1 385.1

Table 10: Query counts with lower, higher or equal recorded time in the adopted pass. Equality uses the stored 0.1-second timing precision.

For example, the SF3 total decreases even though only 22 queries record lower time: the largest reductions outweigh the increases elsewhere. The workload ratio is computed from the two sums of query times, giving each query's elapsed cost its contribution to the total.

6.4 How individual query behavior changes with scale

Q4 shows a consistent lower adopted runtime: from 27.8 versus 16.1 seconds at SF1 to 122.1 versus 71.9 at SF4. Q39 retains an original/adopted ratio between approximately 2.7 and 3.4 across the four sizes.

Q59 has the strongest increase in its recorded ratio among these examples. Original time grows from 25.5 to 334.0 seconds, while adopted time changes from 2.3 to 9.0 seconds. Its ratio therefore grows from 11.09 at SF1 to 37.11 at SF4, making it increasingly important to the larger-scale workload saving.

Q27 shows a different pattern: the adopted pass has higher time at all four scales. Q67's ratio also varies, from below one at SF1–SF3 to 1.12 at SF4. These cases connect the query transformations to their measured behavior under the complete Docker configuration.

The local development studies explain the transformations and their saved plans; the four-scale Docker measurements quantify their behavior in the complete benchmark environment. The preserved SQL snapshots connect each measurement to its executed version.

7 Physical-design and automation work

7.1 Indexing and statistics

The project evaluates index access alongside SQL structure. The current full index emitter selects 166 secondary indexes; the optional pruned recipe selects 95. Primary indexes are additional. The preserved SF4 adopted run records 188 total indexes, and the later original run records 190 following two restored definitions.

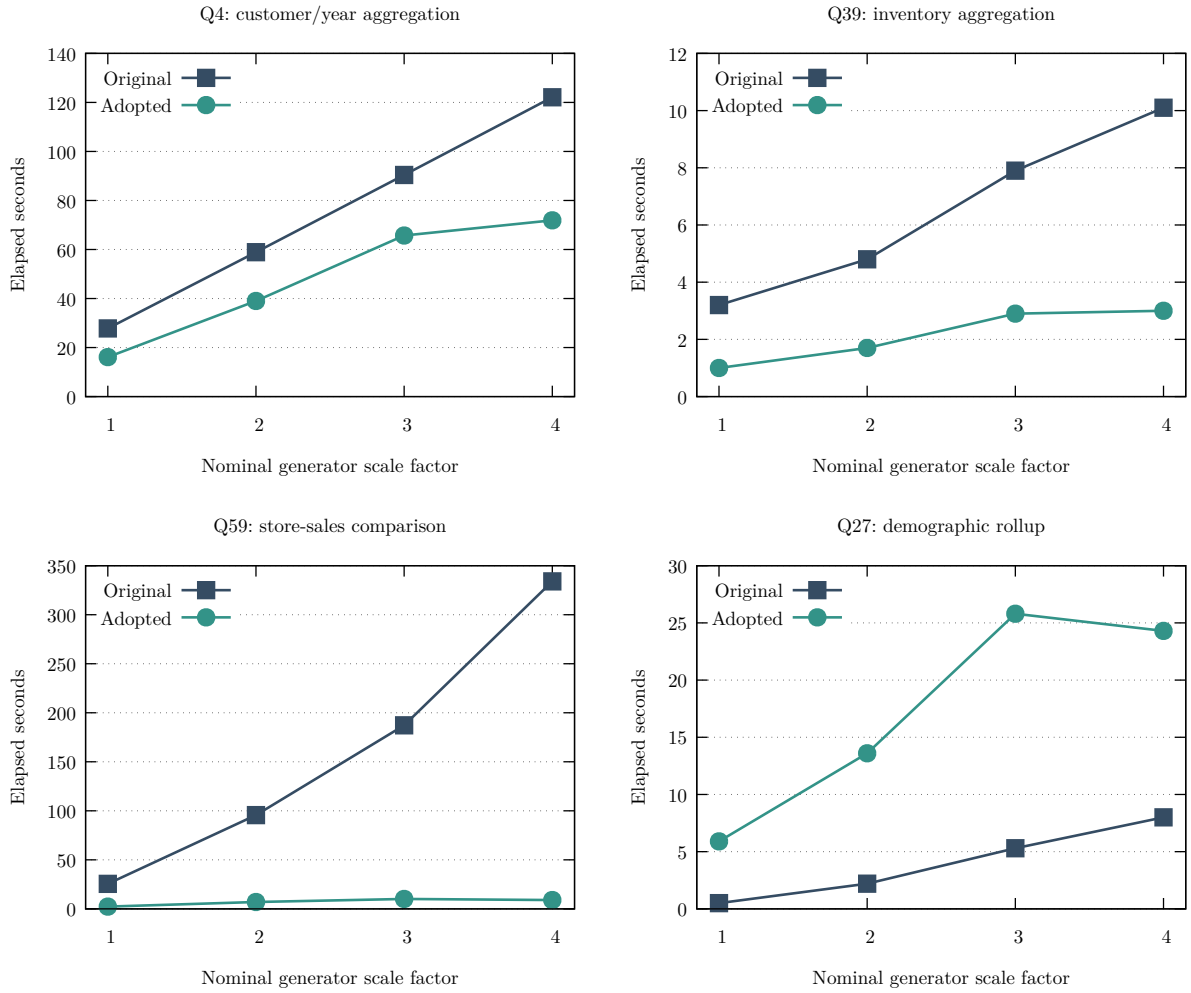


Figure 6: Four representative query-level scaling patterns using the original and adopted versions at each measured size.

Query	SF1	SF2	SF3	SF4
Q4	1.73	1.51	1.38	1.70
Q5	7.00	3.61	2.70	2.92
Q14	10.55	4.23	2.86	4.02
Q23	3.52	2.90	3.03	4.21
Q27	0.08	0.16	0.21	0.33
Q31	1.00	0.73	0.65	0.83
Q39	3.20	2.82	2.72	3.37
Q51	1.23	0.97	1.17	1.35
Q59	11.09	13.66	18.53	37.11
Q67	0.69	0.66	0.62	1.12
Q74	1.03	0.88	0.71	0.96

Table 11: Original/adopted runtime ratios for selected query instances. Values above one indicate lower adopted runtime; values below one indicate higher adopted runtime.

The experiments collect read counters and execution-plan information to study which indexes support the workload. Development reports 28–29 record trials with visible, ignored and dropped indexes, together with changes in statistics and plan choice. Q39 provides a concrete case where selecting a different access path changes runtime substantially.

The design study considers access coverage, selectivity, index width and the cost of building and storing indexes. Read counters describe the paths used during a measured workload. Plan inspection complements those counters by showing the optimizer's selected keys, join order and derived-table operations.

7.2 Reproducible execution workflow

`reproduce.sh` resolves command-line options, environment values and profile defaults, checks Docker memory capacity, builds the image, and starts the container. The source checkout is mounted read-only during execution; data and outputs use separate persistent locations.

The following commands reproduce an SF1 original/adopted pair using the same scale-specific database volume and the group's result-directory convention:

```
./reproduce.sh full SCALE=1 BUFFER_POOL=2G RNGSEED=10 \
  TUNED=tuned QUERYSET=all VOLUME=tpcds-final-sf1 \
  RESULTS_DIR=results/check-original-sf1

./reproduce.sh full SCALE=1 BUFFER_POOL=2G RNGSEED=10 \
  TUNED=tuned QUERYSET=all-tuned VOLUME=tpcds-final-sf1 \
  RESULTS_DIR=results/sf1-all-tuned-v1

python3 -B docker/bench.py --compare \
  results/sf1-all-tuned-v1/tuned \
  results/check-original-sf1/tuned
```

`QUERYSET` chooses the SQL workload; `TUNED` chooses the server settings. The same data volume is reused within each pair. Repeated runs archive earlier outputs, and each scale can use a separate volume. Completed answers and timing artifacts are exported before reclaiming database storage, allowing the experiment to proceed on a machine with limited disk space.

7.3 Report and evidence packaging

The evidence bundles contain successful execution records, complete answers, executed SQL snapshots, summaries and SHA-256 manifests. They remain available independently of the loaded database. The report build reads those artifacts, generates numeric tables and figures, and produces a PDF plus an editable LaTeX package.

```
python3 -B scripts/build_latex_report.py \
  --out submission/latex-report-01
```

The build uses XeLaTeX, BibTeX and gnuplot. It reads completed result pairs from the established directories or preserved evidence bundles. The generated LaTeX ZIP includes data tables, figures, input inventories and the corresponding run evidence. Group names and identifiers are maintained in `report/latex/metadata.tex`. The implementation package contains the code, SQL, toolkit license, configuration and replication instructions.

8 Conclusion

The project combines a complete 99-query MariaDB workload with data generation, loading, physical-design experiments, SQL optimization and automated result comparison. The implemented translations handle MariaDB syntax and preserve hierarchy levels in the corrected rollup queries. Full saved answers support query-by-query comparisons across the measured workloads.

The optimization studies demonstrate several approaches to analytical SQL: filtering early, aggregating periods together, delaying dimension lookups and restructuring window-query inputs. Saved plans and repeated local measurements explain the processing changes behind the selected variants. The complete Docker runs then evaluate the adopted workload with all 99 query identifiers.

The complete campaign evaluates original and adopted workloads at SF1, SF2, SF3 and SF4, comprising 792 query-file executions and 396 answer comparisons. The adopted totals are lower at every scale, with recorded reductions of 25.3%–49.0%. Q14, Q23 and Q59 contribute the largest absolute savings, while the individual query tables show how the effect varies across query structures and sizes.

At SF4, the recorded workload total changes from 2 921.5 to 1 536.4 seconds, with 99 matching ordered query answers. The measured scale pairs, configuration records and full runtime tables connect the report's conclusions to reproducible artifacts. Together, the implementation and experiments illustrate how database design, query structure and execution settings shape analytical performance.

References

[1] Ahmad, Rishika Gupta, Mir Wise Khan, and Chidiebere Ogbuchi. TPC-DS Benchmarking with Postgres Database, 2022. Prior course report supplied as TPC_DS_Benchmarking_Report.pdf; used as a structural reference, not as a source of our measurements.

[2] Nikola Ivanović, Jose Antonio Lorencio Abril, Sayyor Yusupov, and Bogdana Živković. A TPC-DS Benchmark Implementation Using Oracle, 2022. Prior course report supplied as nafiz_report.pdf; used as a structural reference, not as a source of our measurements.

[3] MariaDB. *InnoDB Buffer Pool*, . URL `https://mariadb.com/docs/server/server-usage/storage-engines/innodb/innodb-buffer-pool`. Official documentation, accessed October 2026.

[4] MariaDB. *Derived Table Merge Optimization*, . URL `https://mariadb.com/docs/server/ha-and-performance/optimization-and-tuning/query-optimizations/optimizations-for-derived-tables/derived-table-merge-optimization`. Official documentation, accessed October 2026.

[5] MariaDB. *NULL Values: Ordering*, . URL `https://mariadb.com/docs/server/reference/data-types/null-values`. Official documentation, accessed October 2026.

[6] MariaDB. *Lateral Derived Optimization*, . URL `https://mariadb.com/docs/server/ha-and-performance/optimization-and-tuning/query-optimizations/optimizations-for-derived-tables/lateral-derived-optimization`. Official documentation, accessed October 2026.

[7] Project contributors. TPC-DS on MariaDB: Implementation and Experiment Repository, 2026. URL `https://github.com/hoseinaghaei/mariadb-tpcds`. Executed SQL snapshots and recorded configurations identify individual experiments; repository HEAD is not an execution timestamp.

[8] Sergei Petrunia and contributors. tpcds-run-tool. URL `https://github.com/spetrunia/tpcds-run-tool`. Reference implementation used by the existing project.

[9] Transaction Processing Performance Council. *TPC Benchmark DS: Standard Specification, Version 4.0.0*, November 2024. URL `https://www.tpc.org/TPC_Documents_Current_Versions/pdf/TPC-DS_v4.0.0.pdf`.

A Adopted optimization query IDs

The 35 selected optimization IDs are:

Q2, Q4, Q5, Q7, Q8, Q10, Q11, Q14, Q21, Q22, Q23, Q27, Q31, Q33, Q35, Q37, Q39, Q43, Q46, Q47, Q51, Q54, Q57, Q59, Q65, Q67, Q68, Q70, Q72, Q74, Q78, Q82, Q87, Q88, Q97.

The list is generated from source attribution under `queries/tuned/`. Each ID denotes one adopted benchmark query; development versions and index-only files are recorded separately. Executed SQL hashes preserve the specific versions used in the measurements.

B Full runtime table across the four scales

The table reports seconds for every complete query file at each measured scale. Each original/adopted pair uses the result-comparison procedure in Section 4. The run configurations are those in Table 6.

Table 12: Complete per-query runtimes in seconds at every measured scale.

Query	SF1		SF2		SF3		SF4	
	Original	Adopted	Original	Adopted	Original	Adopted	Original	Adopted
Q1	1.1	1.0	2.2	2.6	3.1	4.0	4.9	4.0
Q2	5.8	2.6	11.0	7.2	16.0	10.6	21.9	9.4
Q3	0.1	0.1	0.1	0.2	0.2	0.2	0.2	0.2
Q4	27.8	16.1	58.9	39.0	90.4	65.7	122.1	71.9
Q5	8.4	1.2	30.7	8.5	41.9	15.5	65.1	22.3
Q6	0.3	0.2	0.5	0.7	0.7	1.0	0.9	1.9
Q7	0.8	2.7	5.8	11.3	8.0	20.5	12.0	26.3
Q8	0.7	0.6	5.0	4.9	6.9	7.4	10.4	10.7
Q9	1.7	1.7	3.4	5.1	5.2	7.8	6.6	6.8
Q10	0.2	0.2	0.3	0.5	0.5	1.0	0.6	0.8
Q11	13.4	11.5	34.4	39.9	50.0	67.6	77.7	75.1
Q12	0.1	0.1	0.1	0.2	0.2	0.2	0.2	0.2
Q13	2.8	2.5	10.7	10.6	15.4	18.6	24.8	22.1
Q14	82.3	7.8	180.1	42.6	280.2	98.0	513.5	127.6
Q15	3.8	0.3	3.9	4.6	5.0	6.1	6.5	6.5
Q16	2.7	0.4	5.4	6.3	8.0	9.4	11.0	10.8
Q17	2.5	1.6	6.6	5.7	13.1	16.9	20.9	18.5
Q18	4.4	3.0	8.0	9.8	10.9	13.4	17.1	16.1
Q19	0.4	0.3	0.5	0.7	0.8	1.1	1.4	1.2
Q20	0.1	0.1	0.2	0.2	0.3	0.3	0.4	0.4
Q21	0.5	0.4	1.0	0.7	1.5	1.2	2.4	1.1
Q22	12.9	7.3	16.4	14.8	27.1	28.1	41.8	28.0
Q23	71.5	20.3	174.2	60.1	262.0	86.5	396.6	94.1
Q24	3.3	6.8	13.4	26.2	10.6	21.2	32.2	48.5
Q25	0.8	2.2	2.0	5.7	5.0	9.5	7.7	11.7
Q26	0.3	0.4	4.1	4.5	6.5	6.2	8.7	8.9
Q27	0.5	5.9	2.2	13.6	5.3	25.8	8.0	24.3
Q28	0.7	0.7	1.1	1.7	1.6	2.4	2.1	2.3
Q29	0.2	0.4	1.4	4.0	3.0	6.5	4.7	7.9
Q30	0.3	0.3	0.5	0.7	0.8	1.0	1.0	1.2
Q31	0.6	0.6	1.1	1.5	1.5	2.3	2.0	2.4
Q32	1.0	1.2	3.0	3.1	4.5	3.7	5.7	5.6

Query	SF1		SF2		SF3		SF4	
	Original	Adopted	Original	Adopted	Original	Adopted	Original	Adopted
Q33	0.1	0.2	0.2	0.4	0.3	0.5	0.4	0.5
Q34	5.2	7.9	11.4	22.8	19.5	35.2	29.0	45.1
Q35	1.8	1.8	2.4	2.7	2.8	3.4	3.4	3.7
Q36	8.3	9.0	16.2	40.8	35.9	76.8	69.2	73.5
Q37	0.1	0.1	0.1	0.1	0.1	0.1	1.4	1.0
Q38	1.6	1.8	3.6	7.3	5.3	8.4	10.8	7.8
Q39	3.2	1.0	4.8	1.7	7.9	2.9	10.1	3.0
Q40	0.4	0.6	1.1	1.3	1.5	1.8	2.5	2.3
Q41	0.7	0.7	1.1	1.7	1.8	2.7	3.9	2.7
Q42	0.2	0.2	0.4	0.6	0.6	1.0	1.4	1.0
Q43	1.0	1.1	1.8	3.2	2.7	4.8	5.5	4.1
Q44	0.0	0.0	0.1	0.0	0.0	0.0	0.1	0.0
Q45	1.5	1.1	3.9	3.9	4.6	5.4	7.8	6.3
Q46	5.8	3.1	13.6	15.2	21.7	23.9	46.6	25.8
Q47	14.8	3.3	31.2	10.6	41.0	15.1	67.4	15.4
Q48	3.1	3.2	5.4	15.1	13.2	23.3	27.6	24.5
Q49	1.5	1.9	13.5	16.1	22.9	30.2	36.7	39.0
Q50	0.1	0.1	0.2	0.3	0.3	1.3	1.3	1.8
Q51	9.8	8.0	22.7	23.3	40.3	34.3	62.1	46.1
Q52	0.2	0.2	0.7	0.6	0.8	1.0	1.1	1.2
Q53	1.3	1.3	3.0	3.7	4.1	5.9	5.6	5.8
Q54	1.3	0.1	2.6	0.2	4.0	0.6	5.9	0.6
Q55	0.2	0.2	0.4	0.6	0.7	1.0	1.1	1.0
Q56	0.3	0.3	0.6	0.8	0.8	1.2	1.2	1.2
Q57	8.6	3.0	19.3	10.6	36.5	20.4	73.6	21.6
Q58	0.2	0.2	0.3	0.4	0.4	0.6	0.7	0.7
Q59	25.5	2.3	95.6	7.0	187.2	10.1	334.0	9.0
Q60	0.7	0.6	1.2	1.8	1.8	2.8	2.7	2.7
Q61	0.0	0.0	0.0	0.1	0.0	0.1	0.0	0.0
Q62	0.8	0.9	1.5	4.6	5.4	7.2	12.5	8.6
Q63	1.3	1.4	2.6	3.8	4.0	5.8	6.9	6.2
Q64	0.1	0.1	0.2	0.3	0.2	0.2	0.3	0.2
Q65	0.9	0.6	2.0	1.8	3.3	3.0	4.8	3.3
Q66	0.3	0.3	0.5	0.7	0.6	0.9	0.9	30.4
Q67	7.6	11.0	21.7	32.8	32.6	52.2	63.1	56.4
Q68	5.2	0.9	17.4	8.3	25.7	12.7	36.0	15.9
Q69	0.7	0.6	1.3	1.6	1.5	1.8	1.6	1.6
Q70	10.8	4.5	54.7	29.5	98.8	53.7	126.3	50.1
Q71	0.9	1.0	10.7	7.1	17.5	17.4	29.8	30.2
Q72	5.7	3.1	10.2	8.4	16.5	14.2	44.3	12.6
Q73	4.6	4.8	15.8	17.6	23.0	30.8	33.9	31.6
Q74	10.0	9.7	29.4	33.3	43.5	61.4	64.1	66.8
Q75	2.7	2.8	6.0	8.6	12.6	16.8	19.7	18.8
Q76	0.9	1.1	8.0	7.3	12.9	14.6	18.5	18.4
Q77	1.3	1.3	8.7	9.2	17.8	20.2	23.7	24.1
Q78	8.8	5.9	19.1	21.7	29.0	30.9	43.3	34.3
Q79	5.2	5.3	13.9	19.0	23.6	34.9	53.4	36.5
Q80	0.7	0.6	1.5	1.8	4.2	4.9	11.2	6.9
Q81	0.3	0.2	0.7	0.9	1.4	1.6	2.2	2.0
Q82	0.1	0.0	0.1	0.1	0.1	0.1	0.9	0.1
Q83	0.1	0.1	0.5	0.6	0.8	1.0	1.8	1.2
Q84	0.2	0.1	0.2	0.3	0.3	0.4	0.5	0.4
Q85	0.2	0.2	1.8	2.2	2.2	3.3	6.8	4.0

Query	SF1		SF2		SF3		SF4	
	Original	Adopted	Original	Adopted	Original	Adopted	Original	Adopted
Q86	1.4	1.4	2.7	4.2	4.1	6.3	6.0	6.1
Q87	1.7	0.7	3.5	1.8	5.4	2.7	7.8	2.8
Q88	0.6	0.2	1.0	0.4	1.1	0.4	1.9	0.5
Q89	1.6	1.5	3.0	4.6	4.8	7.2	6.9	7.0
Q90	0.8	0.8	1.6	4.1	3.0	10.2	9.6	11.9
Q91	0.1	0.1	0.4	0.3	0.4	0.8	0.8	0.9
Q92	0.1	0.1	0.4	0.3	1.3	1.6	4.1	3.0
Q93	0.1	0.1	1.2	1.3	2.5	3.1	3.6	3.7
Q94	0.0	0.0	0.1	0.1	0.1	0.1	0.2	0.1
Q95	0.0	0.0	0.0	0.1	0.0	0.1	0.1	0.1
Q96	0.1	0.1	0.1	0.1	0.1	0.2	0.2	0.2
Q97	2.5	1.5	5.2	5.2	7.9	6.8	11.1	6.8
Q98	0.2	0.2	0.3	0.4	0.5	0.6	0.6	0.6
Q99	3.3	2.9	8.8	10.5	11.7	13.4	17.9	15.9
Total	431.4	219.9	1 102.4	785.0	1 765.3	1 318.0	2 921.5	1 536.4

C Implementation and evidence map

Component	Repository files
Schema and relationships	`sql/01_schema.sql`, `sql/02_foreign_keys.sql`; reports 04, 06, 07 and 14
Generation and loading	`docker/entrypoint.sh`, `docker/profiles.sh`, `sql/04_load_data.sql`
Adaptation and rollup checks	`report/10-adapt-queries.md`, `queries/manual-fixes/`, `scripts/test_rollup_correctness.py`
Complete execution and comparison	`docker/bench.py`, `docker/result_sets.py`, `scripts/test_benchmark_correctness.py`
Four-scale Docker evidence	`experiments/evidence/docker_sf1_99/` through `experiments/evidence/docker_sf4_99/`
Repeated local studies	`experiments/evidence/riccio_sf1_20261014/`, `queries/optimized/`
Q4/Q67 follow-up	`report/27-query-followup.md`, `experiments/evidence/slow_queries_20261027/`
Report inputs	`generated/evidence.json`, `SOURCE-MANIFEST.json` in the LaTeX ZIP